Branch Coverage Report — Pizza Order

Lab 6: Code and Branch Coverage Analysis | Application: pizza_order.py (SETAPESU26/L6_L7_24_pizza_order, Lab 6)

Student: Akshat Agarwal, PES1UG24AM024 | Repository: github.com/Akshat1686/L6_pizza_order_024

Environment verified: Python 3.12.3 / pytest 9.2.0 / coverage 7.17.0 on Linux, and Python 3.12.10 / pytest 9.2.0 on Akshat's Windows terminal (screenshots in this report). The application file pizza_order.py was not changed; only test files were added.

1. What branch coverage means, and why line coverage is not enough

Branch coverage asks, for every if / elif in the code, did the condition evaluate to True at least once AND to False at least once? A branch that is only ever taken one way is a 'partial branch', and coverage.py reports it separately (BrPart). The handout's calculator example makes the trap concrete: a single test with a member buying a 150 item executes every line of calculate_discount, giving 100% line coverage, while never testing is_member = False or price <= 100 — so its branch coverage is partial. The same trap exists in pizza_order.py: a suite that only ever ordered cheese-burst pizzas would execute the 'price += 100' line while never proving the no-cheese-burst path, and a suite that only tested students would never prove the non-student path.

The lab's pass mark is at least 90% branch coverage. This suite reaches 100% — 24 branches, 0 partial.

2. Application under test and its 24 branches

pizza_order.py has 12 decision points. Each if contributes two branches (True/False), which is how 12 decisions become 24 branches in coverage.py's count. No function was edited for this lab.

Function	Decision	True outcome	False outcome
pizza_price	size not in SIZE_PRICE	raise (line 6)	continue pricing
pizza_price	toppings < 0	raise (line 8)	continue pricing
pizza_price	toppings > 3	120 + 30 per extra (line 11)	40 per topping (line 13)
pizza_price	is_cheese_burst	check size, add 100 (line 17)	skip cheese burst
pizza_price	size == 'small' (inside cheese burst)	raise (line 16)	add 100
order_discount	order_total < 300	return 0 at once (line 23)	compute a day discount
order_discount	day == 'tuesday'	half the total (line 25)	check weekend
order_discount	day in (saturday, sunday)	discount 0 (line 27)	tenth of total (line 29)

order_discount	is_student	discount += 50 (line 31)	discount unchanged
delivery_status	distance_km > 10	'not deliverable' (line 37)	check time
delivery_status	minutes_taken <= 30	'on time' (line 39)	check 45-minute bound
delivery_status	minutes_taken <= 45	'late' (line 41)	'free pizza' (line 42)

3. Starting point

With only the template's two starter tests, branch coverage of pizza_order.py was 34%: 24 branches existed, 6 were partial, and 20 statements never ran. Every discount branch, every error branch, and the late / free-pizza paths were untested in either sense.

4. The branch-coverage test suite

File: BranchCoverageTest_024_pizza_order.py (024 = PES1UG24AM024). 25 tests. Tests 1–20 are the code-coverage suite, unchanged — they already take both outcomes of most decisions. Tests 21–25 are the branch additions (marked BRANCH-ADD in the file): they exist specifically to force the remaining outcomes and the boundary values where an off-by-one error (< vs <=) would otherwise go unnoticed.

4.1 Tests 1–20 — the core suite (summarised)

pizza_price (9 tests): plain medium 350; small with 2 toppings 280; large with 3 toppings 620 (top of the <=3 tier); large with 5 toppings 680 (>3 tier); medium cheese burst 530; large cheese burst with no toppings 600; unknown size, negative toppings, and cheese-burst-on-small all raise ValueError. order_discount (7 tests): under-300 total returns 0 even on Tuesday; Tuesday 400 gives 200, and 250 with student; weekend 400 gives 0, and 500 on Sunday with student gives 50; weekday 400 gives 40, and 90 with student. delivery_status (4 tests): 20 min / 11 km is not deliverable; 20 min / 3 km is on time; 40 min is late; 60 min is free pizza. Full input/expected pairs are in the companion Code Coverage Report, §4.

4.2 Tests 21–25 — the branch additions, in detail

#	Test	Assertions	Branch / boundary it forces
21	test_branch_toppings_boundary_four_vs_three	large, 4, False -> 650; small, 0, False -> 200	toppings > 3 at exactly 4 (one discounted topping: 500 + 120 + 30), the smallest >3 case; and is_cheese_burst = False explicitly, which line coverage alone can hide because there is no else-line to execute
22	test_branch_discount_exactly_300_not_below_minimum	300, monday, False -> 30; 300, tuesday, False -> 150	order_total < 300 = False at exactly 300 — the discount must still apply at the boundary, not return 0
23	test_branch_discount_below_minimum_ignores_stud	299, monday, True -> 0	The early return must ignore the student

	ent		surcharge: 299 with a student is still 0, not 50
24	test_branch_delivery_time_boundaries	30 min -> on time; 31 min -> late; 45 min -> late; 46 min -> free pizza	Both sides of both time bounds: <=30 True at 30 and False at 31; <=45 True at 45 and False at 46
25	test_branch_delivery_distance_boundary	20 min, 10 km -> on time; 60 min, 10 km -> free pizza	distance_km > 10 = False at exactly 10 km — still deliverable, unlike 11 km

5. Branch-by-branch evidence

Every decision below was observed in both outcomes during the branch run. This is the table coverage.py's '0 partial branches' summarises.

Decision	True — shown by	False — shown by
size not in SIZE_PRICE	unknown size raises (test 7)	every valid-size test
toppings < 0	negative toppings raise (test 8)	every non-negative test
toppings > 3	large 5 toppings (test 4); large 4 (test 21)	large 3 toppings (test 3)
is_cheese_burst	medium / large cheese burst (tests 5, 6)	plain medium (test 1); small 0 (test 21)
size == small (cheese)	cheese burst on small raises (test 9)	cheese burst on medium / large (tests 5, 6)
order_total < 300	250 returns 0 (test 10); 299 (test 23)	400 / 500 totals; exactly 300 (test 22)
day == tuesday	400 Tuesday (tests 11, 12)	Saturday / Monday tests
day in weekend	Saturday / Sunday (tests 13, 14)	Monday / Wednesday (tests 15, 16)
is_student	student tests (12, 14, 16, 23)	non-student tests (11, 13, 15, 22)
distance_km > 10	11 km not deliverable (test 17)	3 / 5 / 10 km tests; exactly 10 (test 25)
minutes_taken <= 30	20 / 30 min on time (tests 18, 24)	31 / 40 / 46 / 60 min tests
minutes_taken <= 45	40 / 45 min late (tests 19, 24)	46 / 60 min free pizza (tests 20, 24)

6. Results

Commands run (from the folder containing pizza_order.py):

```
coverage run --branch -m pytest BranchCoverageTest_024_pizza_order.py -v

coverage report -m
```

File	Statements	Missed	Branches	Partial	Coverage
BranchCoverageTest_024_pizza_order.py	61	0	0	0	100%
pizza_order.py	34	0	24	0	100%
TOTAL	95	0	24	0	100%

All 25 tests passed, in 0.09 s on the Linux verification run and in Akshat's own Windows run (Python 3.12.10, pytest 9.2.0, 25 passed). The single warning in the Windows run is the same unrelated anyio/pytest CallSpec2 deprecation notice as in the code-coverage run; it is not a

failure. The lab requirement (>=90% branch) is met with 10 points of headroom, and under line measurement the same suite is also 100%.

7. Screenshots (Akshat's Windows terminal)

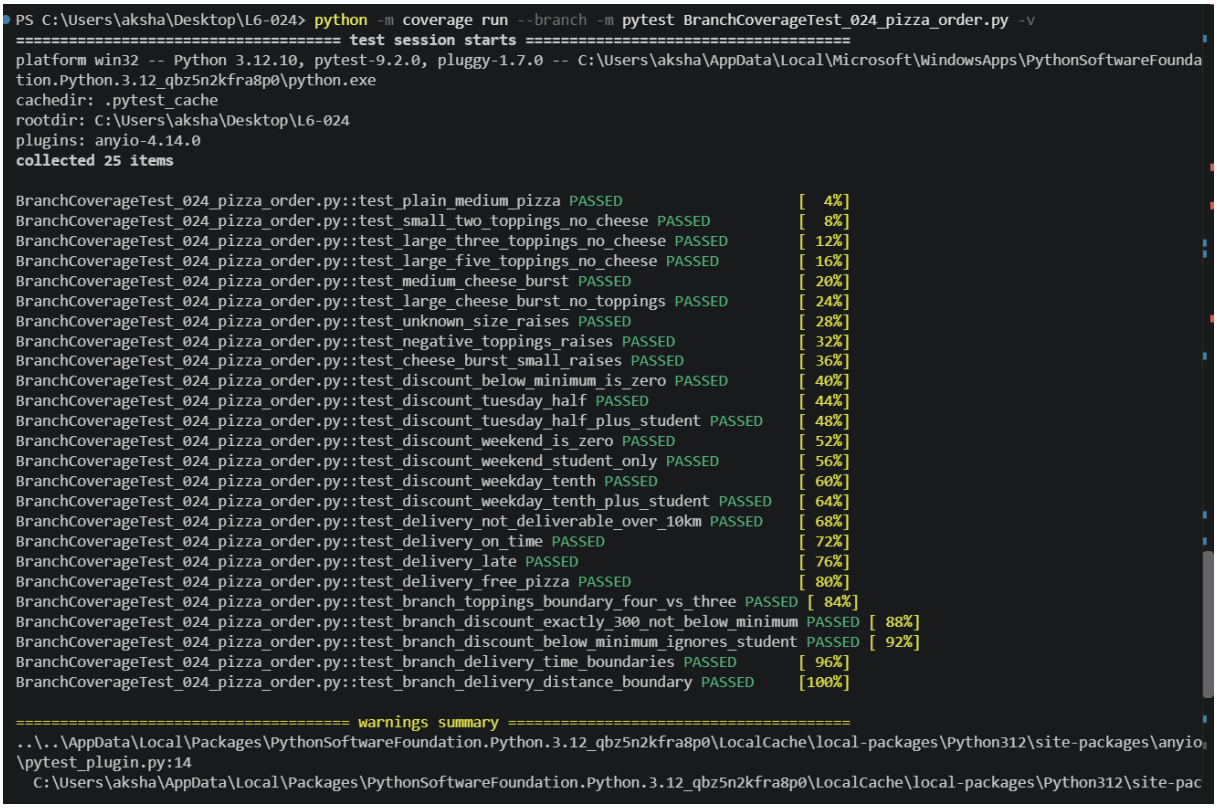


Figure 1 — Branch-coverage test run: 25 tests, all PASSED.

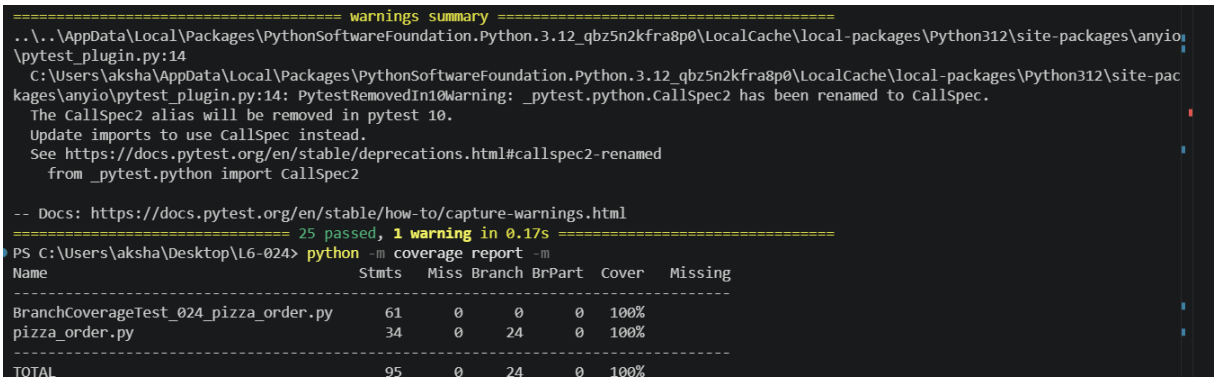


Figure 2 — Branch-coverage report: pizza_order.py 34 statements, 24 branches, 0 partial, 100%.

8. Interpretation and limits

- What 100% branch coverage proves: no decision in pizza_order.py was only ever exercised one way, and the boundary values around every threshold (3/4 toppings, 299/300 total,

30/31 and 45/46 minutes, 10/11 km) behave as specified. An off-by-one change to any threshold would now fail a test.

- What it does not prove: path coverage. Branch coverage tests each decision both ways, but not every combination of decisions (for example, cheese burst combined with every topping count). The tested combinations are the realistic ones for this application, but they are not exhaustive.
- What it does not prove either: correctness for untested inputs. Coverage measures execution paths, not a proof that the pricing or discount rules are the right business rules — only that the code does what its tests assert.

Conclusion: the branch-coverage deliverable is complete — 25 tests, 100% line and 100% branch coverage, 0 partial branches, all tests passing, with terminal screenshots as evidence.